

Table of Contents

Microcontroller (MCU) overview	3
Microcontroller (MCU) definition	3
Fundamental concept: 1. Flash	3
2. Ram.....	3
3. EEPROM	3
4. Registers	3
5. GPIO	4
6. Clock.....	4
7. Interrupt	4
8. Fundamental flow	4
Pic18f45k50.....	5
1. Device Overview	5
2. Pin Diagram	6
3. Memory Organization.....	9
4. Oscillator	9
5. I/O Ports	9
6. Configuration Bits	10
7. Flow.....	10
8. Practice	11
8.1. Tools	11
8.2. Project structure	11
8.3. Configuration Bits	12
8.4. Write the program.....	12
8.5. Build the project	13
8.6. Debugging	16
DSP56F80x	18
1. Device Overview	18
2. Pin Diagram	19
3. Memory Organization.....	20
4. Oscillator / Clock.....	20
5. GPIO / I/O Ports.....	21
6. Startup hardware settings	21
7. DSP Core	22
7.1. Multiply-Accumulate (MAC) Unit.....	22
7.2. 36-bit Accumulators	22
7.3. Three 16-bit Input Registers	22
7.4. Address Generation Unit (AGU).....	23
7.5. Hardware Loop Unit.....	23
8. Practice	23

8.1. Development Environment.....	23
8.2. Project structure	23
8.3. Write the program.....	27
8.4 Build, Debugging, and Memory Verification.....	35

Microcontroller (MCU) overview

Microcontroller (MCU) definition

A microcontroller is a tiny, self-contained computer on a single integrated circuit designed to manage specific tasks in embedded systems.

For a pc, it has these components: CPU, RAM, SSD, GPU, USB, Keyboard, Display. Each is separated hardware. A microcontroller is: CPU + Flash, RAM, GPIO, Timers, UART, ADC, PWM, Interrupt Controller, Clock. Everything is inside one chip. Unlike pc, a microcontroller has no operating system, program starts running immediately after reset.

A microcontroller is mostly used in applications that need to read input from sensors, processes data, and triggers actions with very low latency, low power and low cost. Common applications include home appliances like washing machines, car engine systems, medical tools like heart monitors, and internet-connected smart devices, ...

Fundamental concept:

1. Flash

Flash is non-volatile memory that when turn off, it does not lose data. Flash stores program code and sometimes constant data (such as lookup tables or constant strings). Unlike on pc, CPU can execute directly from flash, the program is not copied into RAM before running.

2. Ram

Ram is volatile memory that when turn off, it loses data. Ram has fast reads/writes. RAM is mainly used for: variables, stack, function, parameters, return addresses, buffers.

3. EEPROM

EEPROM is non-volatile memory that is limited in storage so it should only be used to save settings.

Why not everything needs non-volatile memory just save on flash instead?

- Flash has limited erase granularity: MCU often has to read, erase and rewrite entire memory if something changes while EEPROM allows updating small values much more easily.
- Flash has limited write endurance with 10,000 erase cycles compared to EEPROM 100,000+
- Flash usually stores your firmware so overwrite random Flash locations can destroy part of the program.
- Programming Flash is disruptive: Some MCUs cannot execute code from Flash while writing Flash.

4. Registers

A microcontroller (MCU) register is a tiny, high-speed storage location inside the processor (CPU registers) or peripheral blocks (memory-mapped peripheral registers). It holds bits that the

CPU uses to temporarily store data, execute math, track memory addresses, or directly control hardware features like timers, pins, and communication ports.

Different microcontrollers (MCUs) have different registers. Different registers in a microcontroller (MCU) control different hardware features, though some registers work together to manage the same feature or handle general CPU tasks.

5. GPIO

GPIO stands for General-Purpose Input/Output. It is a flexible digital pin on a microcontroller (MCU) whose function whether it reads an incoming signal (input) or sends an outgoing signal (output) is completely defined by software.

6. Clock

6.1. Oscillator Clock (Source Clock)

The raw, rhythmic periodic electrical signal generated by a physical hardware component (such as an internal RC circuit or an external quartz crystal). It acts as the fundamental timebase or “heartbeat” for the microcontroller.

6.2. CPU Clock (System Clock $F_{\{OSC\}}$)

The final, conditioned periodic signal delivered directly to the microprocessor core to drive the execution of instructions and state machine transitions. It determines the exact speed at which the CPU processes software commands.

Each clock cycle advances the CPU. Depending on the instruction and architecture, an instruction may require one or more clock cycles to complete.

Every instruction requires clock cycles. => No clock. No program. Delay command is meaningless.

7. Interrupt

A microcontroller unit (MCU) interrupt is a hardware or software signal that forces the CPU to pause its current program, save its current state, and instantly run a special function called an Interrupt Service Routine (ISR) to handle an urgent event before resuming normal work.

Compare to polling:

Unlike interrupt, polling is the process where it repeatedly checks and updates data in a define timed loop.

8. Fundamental flow

```
Write C code
↓
XC8 Compiler
↓
Machine Code
↓
.hex file
↓
Flash Memory
```

↓
CPU fetches instruction
↓
Registers change
↓
GPIO changes voltage
↓
LED turns on

9. Memory-mapped I/O

Memory-mapped I/O is a hardware design where peripheral control registers are assigned addresses within the CPU's address space. The CPU accesses hardware peripherals using the same load and store instructions it uses for memory, without requiring special instructions for each peripheral.

Compare to port-mapped I/O:

Port-mapped I/O is a way for a computer's central processing unit to talk to extra parts like screens or keyboards using a separate address space and special instructions like IN and OUT. The processor must use specific instructions (such as IN and OUT) instead of normal read and write memory commands. The downside is it requires specialized assembly instructions. Data movement is often limited to small chunks (like bytes or words), making heavy data transfers slower than memory-mapped I/O. The only upside is that each peripheral has its own address space so it saves CPU address space which can be used for RAM however if CPU address space is more than enough then there is no benefit.

⇒ Most modern microcontrollers (MCUs) use memory-mapped I/O.

Pic18f45k50

Document Source: <https://ww1.microchip.com/downloads/en/devicedoc/30000684B.pdf>

1. Device Overview

The PIC18F45K50 is a low-power, high-performance 8-bit microcontroller with:

- 32 KB Flash Program Memory
- 2 KB SRAM
- 256 B EEPROM
- Up to 36 GPIO pins
- USB 2.0
- Timers
- Interrupts
- UART
- SPI
- I2C
- ADC

- PWM
- Comparators
- Internal and external oscillator options
- Operating frequency up to 48 MHz with PLL
- Extreme Low-Power Management with XLP

PIC18F45K50 serves as a modern, cost-effective drop-in replacement for older legacy chips like the PIC18F4550.

Common applications:

USB Interface Devices: Custom human interface devices (HID) like specialized keyboards, game controllers, and custom mice.

Industrial Automation: Data logging systems, sensor monitors, and localized control panels using its 10-bit ADC channels.

Educational & Hobbyist Projects: Microcontroller training boards for learning embedded C programming.

Battery-Powered Gadgets: Portable instruments leveraging its nanoWatt XLP power-saving sleep modes.

2. Pin Diagram

Example pin RB0 can be: Digital Input, Digital Output, External Interrupt, SPI SDA, Analog Input based on TABLE 1: PIC18(L)F2X/45K50 PIN SUMMARY in page 5. Depends on the purpose to choose which pin to be used for a specific function the pin can serve.

Functions:

Analog: AN0, AN1, AN2,...

Means

Can connect to ADC.

Example

Potentiometer

↓

ADC

↓

Digital number

Comparator

Instead of converting voltage,

Comparator simply answers

Voltage A >

Voltage B ?

Output

YES

NO

Useful for threshold detection.

CTMU

Charge Time Measurement Unit.

Used for

- touch sensing
- capacitance measurement

SR Latch

Hardware memory.

Stores 0 or 1 without CPU involvement.

Reference

Voltage Reference. Provides stable voltages

USB

Means this pin participates in USB communication.

Example

USB D+

USB D-

(E)CCP

Enhanced Capture Compare PWM.

EUSART

Serial communication.

Example

TX

↓

Send bytes

RX

↓

Receive bytes

MSSP

Master Synchronous Serial Port.

Hardware supports: SPI or I2C

Timers

Can be connected to Timer0, Timer1, Timer3, ...

Useful for

- delays
- counting
- frequency measurement

Interrupts

Example

Button

↓

RB0

↓

Interrupt

↓

CPU wakes immediately

Pull-up

Means the MCU has an internal resistor.

Example

VCC

↓

10k resistor

↓

Button

↓

GND

Basic

Means normal GPIO. Just Input or Output

ICD

In-Circuit Debugger connects a host computer to a target microcontroller, allows developers to test, run, step through code, set breakpoints, and inspect registers or variables directly on real physical hardware.

3. Memory Organization

The datasheet lists the PIC18F45K50 as having 32 KB Program Memory, 2 KB SRAM, and 256 bytes of Data EEPROM.

Flash for program, RAM for variables, EEPROM for saved settings, Registers to control hardware. Base on that to optimize memory to stay within memory constraint.

4. Oscillator

The datasheet says this family supports:

- Internal oscillator (31 kHz–16 MHz)
- External crystal
- External clock
- PLL up to 48 MHz
- Fail-Safe Clock Monitor
- Two-Speed Start-up
- Active Clock Tuning
- ⇒ Can add external crystal for extreme precision or just use internal oscillator
- ⇒ PLL up to 48 MHz so The CPU runs faster without changing the crystal. Supports configuration for both 3x and 4x multiplication. => 16MHz internal or external crystal input => 48 MHz CPU
- ⇒ Fail-Safe Clock automatically switch to internal oscillator if crystal fails to ensure the program keeps running

5. I/O Ports

This is where GPIO lives. The block diagram shows several ports: PORTA, PORTB, PORTC, PORTD, PORTE. Each port contains multiple pins.

For each pin there three registers will likely to be used:
TRIS -> Direction, PORT -> Read, LAT -> Write

Why LAT instead of PORT?

The PIC provides LAT as an output latch. Writing to LAT updates the output value, while PORT reflects the actual electrical level on the pin. Using LAT for outputs avoids subtle read-modify-write issues.

6. Configuration Bits

Configuration bits are startup hardware settings. They are programmed into a dedicated configuration memory area before the MCU ever executes `main()`.

Flow:

Power On



Read Config Bits



Configure Hardware



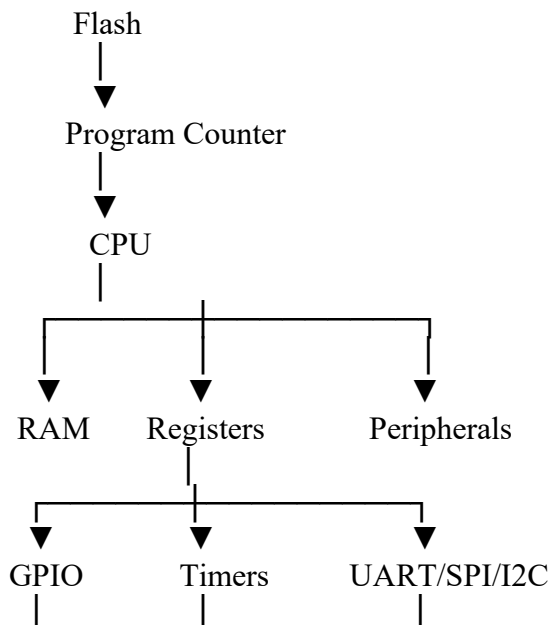
`main()`

Examples

- **Oscillator:** Internal or External
- **Watchdog:** ON => **auto reset** if firmware freezes
- **Brown-Out Reset:** Voltage drops => Reset
- **Low Voltage Programming:** Enable => Can program without high voltage
- **Power-Up Timer:** Power applied => Wait => Start CPU

7. Flow

Simplified diagram in page 13 FIGURE 1-1: PIC18(L)F2X/45K50 FAMILY BLOCK DIAGRAM:



▼ ▼ ▼
LEDs Delays Communication

The diagram above illustrates the basic processing flow of a microcontroller:

1. The CPU reads instructions from Flash memory using Program Counter.
2. During execution, the CPU uses RAM to store temporary data and interacts with registers to control the hardware.
3. These registers control peripherals such as GPIO, timers, and communication interfaces (UART, SPI, I²C).
4. Finally, the peripherals interact with the outside world—for example, by toggling LEDs, generating delays, or transmitting data between devices.

8. Practice

Demo program: Write an LED Blink Program

8.1. Tools

MPLAB X IDE 6.35, XC8 compiler version 3.10. Hardware tool: Simulator.

8.2. Project structure

Project name: PIC18

⇒ Project structure:

```
PIC18/
|
|— nbproject/
|— build/
|— dist/
|— main.c
|— Makefile
```

main.c

This contains firmware. Execution begins in:

```
void main(void)
{
}
}
```

nbproject/

This is MPLAB's project configuration.

build/

Temporary compilation files. Will be recreated after cleaning the project.

dist/

It contains the final build artifacts to be used or distributed.

Makefile

The Makefile defines how source files become firmware.

8.3. Configuration Bits

Configuration bits are special settings stored in dedicated configuration memory, separate from your normal program code.

They tell the microcontroller how to start before `main()` runs.

Typical options include:

- Oscillator source (internal or external)
- Watchdog Timer enable/disable
- Power-up Timer
- Brown-out Reset
- Low-voltage programming

For this demo program, set Watchdog Timer disable (unexpected resets can be confusing for a demo) and Oscillator internal with ‘`#pragma config`’ syntax:

```
#pragma config WDTEN = OFF
```

```
#pragma config FOSC = INTOSCIO
```

8.4. Write the program

`main.c`:

```
#include <xc.h>
```

```
// Example configuration bits
```

```
#pragma config WDTEN = OFF
```

```
#pragma config FOSC = INTOSCIO
```

```
#define _XTAL_FREQ 16000000UL // Set as Unsigned Long to prevent compiler math  
truncation
```

```
static void oscillator_init(void)
```

```
{
```

```
    // Force the internal oscillator to run at 16 MHz
```

```
    OSCCONbits.IRCF = 7;
```

```
}
```

```
void main(void)
```

```
{
```

```
    oscillator_init();
```

```
    TRISBbits.TRISB0 = 0; // Make RB0 an output
```

```
    while (1)
```

```
    {
```

```
        LATBbits.LATB0 = 1;
```

```
        __delay_ms(500);
```

```
        LATBbits.LATB0 = 0;
```

```

    __delay_ms(500);
}
}

```

For simple LED project, CPU can run at very low frequency all the way down to 31.25kHz with noticeable power saving benefit but for testing purpose only, 16MHz is great for benchmarking.

#define _XTAL_FREQ 16000000UL: Define the system oscillator frequency is 16MHz. It only tells the software math what speed the chip is running at, it does not change actual chip clock registers. Therefore, OSCCON register need to be set for internal oscillator to run at 16 MHz: OSCCONbits.IRCF = 7;

based on **Register Definitions: Oscillator Control** in page 33

TRISBbits.TRISB0 = 0; configures RB0 as an output.

LATBbits.LATB0 = 1; drives the pin high.

LATBbits.LATB0 = 0; drives the pin low.

delay_ms(500); pauses execution. The compiler calculates the delay using _XTAL_FREQ.

Active-High Setup

Wiring: Connect RB0 to a resistor, then to the LED's anode (positive side), and the cathode (negative side) to Ground (GND).

LATBbits.LATB0 = 1 sends 5V/3.3V out of the pin, which turns the LED ON.

LATBbits.LATB0 = 0 drops the pin to 0V, which turns the LED OFF

Active-Low Setup (Standard)

Wiring: Connect VDD (5V/3.3V) to a resistor, then to the LED's anode, and the cathode to RB0.

LATBbits.LATB0 = 1 matches the VDD voltage, stopping current flow, which turns the LED OFF.

LATBbits.LATB0 = 0 creates a path to Ground, which turns the LED ON.

8.5. Build the project

- Build result:

18F45K50 Memory Summary:

Program space	used	54h (84)	of	8000h bytes	(0.3%)
Data space	used	2h (2)	of	800h bytes	(0.1%)
Configuration bits	used	2h (2)	of	7h words	(28.6%)
EEPROM space	used	0h (0)	of	100h bytes	(0.0%)
ID Location space	used	0h (0)	of	8h bytes	(0.0%)

Program space used 84/ 32,768 decimal bytes of flash memory

Data space used 2/2,048 decimal bytes of

Configuration bits used 2/7 words

EEPROM space used 0/256 decimal bytes
ID Location space used 0/8 decimal bytes

⇒ No memory issue.

- Build output:

Key files:

.hex

This is the file that gets programmed into the microcontroller. It contains machine code in a text format understood by programmers such as a PICKit. Can be considered as the firmware image.

.elf

This contains the machine code plus debugging information. Debuggers use it to relate machine instructions back to C source, variables, and functions

.map

This is generated by the linker.

It answers questions like:

- How much Flash did the program use?
- How much RAM did variables use?
- Where was each function placed in memory?
- Where is `main()` located?

⇒ To understand memory usage and diagnosing issues.

Key elements:

- Program space

SEGMENTS

reset_vec	000000	Length 000004
text0	000802	Length 00004A
text1	000846	Length 000006
init	007FFC	Length 000004

⇒

reset_vec	4 bytes
text0	74 bytes
text1	6 bytes
init	4 bytes

Total 88 bytes

Close to the reported 84 bytes Program space. The slight difference is due to how the linker organizes code sections in Flash.

- Data space used 2 bytes:
cstackCOMRAM 000001
Length 000002
=>
Address 0x0001
↓
2 bytes reserved

The program has no variable, this is compiler-generated stack space.

- Module information

main.c

_oscillator_init
CODE 0846 Size 6

_main
CODE 0802 Size 62

Meaning:

Function	Address	Size
_main	0x0802	62 bytes
_oscillator_init	0x0846	6 bytes

_main text0 000802

Meaning:
Flash Address 0x0802

- Registers:
_LATBbits (abs) 000F8A

_TRISBbits (abs) 000F93

_OSCCONbits (abs) 000FD3

⇒ Register has fixed hardware addresses. The compiler generates an instruction that writes to address. That's memory-mapped I/O in action.

- Reset vector:

reset_vec 000000 Length 4

Every time the PIC powers on:

CPU

↓

Address 0x0000

↓

reset vector

↓

startup code

↓

main()

- text0:

text0 000802 Length 4A

Machine instructions generated from C code => entire main() function lives there.

- text1:

text1 0x846

contains: oscillator_init()

- init:

init 0x7FFC

Startup code. XC8 performs work like: initialize RAM, initialize stack, initialize constants, prepare runtime...

- Unused Address Ranges

CODE 00084C-007FFB

Meaning: Everything here is unused Flash. Range from 0x00084C to 0x007FFB spans a total size of 0x77AF bytes (30,639 decimal bytes or roughly 29.92 KB)

8.6. Debugging

Set breakpoint is the first line of main(): oscillator_init()

Start debugging with SFR Window opened, observe registers value change when step into:

OSCCONbits.IRCF = 7;

OSCCON 0x30 48 00110000 (IRCF bits = 101)

=> 0x70 112 01110000 (IRCF bits = 111)

Internal oscillator run at 4MHz to 16MHz

TRISBbits.TRISB0 = 0;

```
TRISB 0xFF 255 11111111
=> 0xFE 254 11111110
Pin RB0 from default input to output
```

```
LATBbits.LATB0 = 1;
LATB 0x00 0 00000000
=> 0x01 1 00000001
Drives the pin high.
```

```
LATBbits.LATB0 = 0; drives the pin low.
LATB 0x01 1 00000001
=> 0x00 0 00000000
Drives the pin low.
TMTM;
```

Execute Delay: The compiler generated many instructions, when step into it means stepping through the whole delay implementation.

Program counter: For 8 bit MCU, only PCL (PCL (Program Counter Low): Contains bits 0 to 7) is a physically mapped memory register.

main() from 0x0 to 0x2E

oscillator_init() from 0x46 to 0x4A

PCL value increases by 2 on every basic instruction step means the CPU is incrementing its

Program Counter pointer to point to the next execution address in Flash memory correctly. Open Disassembly to observe instructions:

```
!void main(void)
!{
! oscillator_init();
! TRISBbits.TRISB0 = 0; // Make RB0 an output
0x806: BCF TRISB, 0, ACCESS
! while (1)
! {
! LATBbits.LATB0 = 1;
0x808: BSF LATB, 0, ACCESS
! __delay_ms(500);
0x80A: MOVLW 0xB
0x80C: MOVWF __rparam_used, ACCESS
0x80E: MOVLW 0x26
0x810: MOVWF __pcstackCOMRAM, ACCESS
0x812: MOVLW 0x5E
0x814: DECFSZ WREG, F, ACCESS
0x816: BRA 0x814
0x818: DECFSZ __pcstackCOMRAM, F, ACCESS
0x81A: BRA 0x814
0x81C: DECFSZ __rparam_used, F, ACCESS
0x81E: BRA 0x814
```

```

!    LATBbits.LATB0 = 0;
0x820: BCF LATB, 0, ACCESS
!    __delay_ms(500);
0x822: MOVLW 0xB

!static void oscillator_init(void)
!{
!    // Force the internal oscillator to run at 16 MHz
!    OSCCONbits.IRCF = 7;
!}
0x84A: RETURN 0

```

⇒ Simulation matches expectations, .hex file is ready to flash onto the real PIC18F45K50.

DSP56F80x

Document Source: <https://community.nxp.com/pwmxy87654/attachments/pwmxy87654/cw-classic/7920/1/DSP56F801-7UM-R5.pdf>

1. Device Overview

The DSP56F80x is not just a microcontroller like PIC18F45K50.

Motorola calls it a **Hybrid Controller** because it combines:

- A microcontroller (GPIO, timers, UART, Flash, interrupts)
- A DSP (Digital Signal Processor) capable of fast mathematical operations
- Motor-control peripherals (PWM, ADC, Quadrature Decoder)

Unlike the PIC18 which is designed for general embedded tasks, this chip is designed for:
Motor Control

- Action: Regulates motor speed, torque, and position.
- How: Reads current sensors, calculates vector control math, and outputs precise PWM signals to drive 3-phase electric motors smoothly.

Industrial Control

- Action: Manages factory automation and robotics.
- How: Interfaces with encoders, monitors temperature sensors, executes PID loops, and sends data over rugged networks like CAN bus.

Digital Power

- Action: Controls voltage and power efficiency.
- How: Samples high-speed analog signals to adjust switch-mode power supplies (SMPS) and runs algorithms for Active Power Factor Correction (PFC).

Audio Processing

- Action: Modifies and enhances sound signals.

- How: Runs real-time math filters to perform equalization (EQ), reduce background noise, compress dynamic range, or create digital sound effects.

Signal Processing

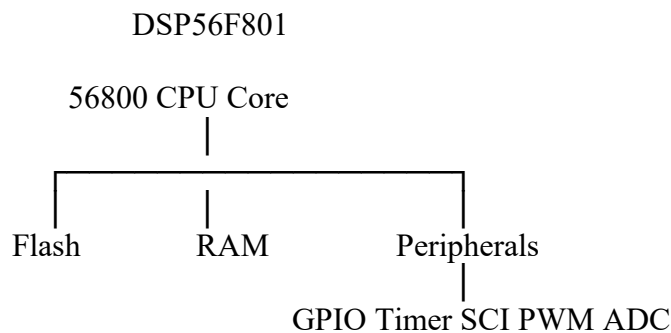
- Action: Extracts useful data from raw sensor streams.
- How: Executes Fast Fourier Transforms (FFT) and digital filters to remove noise from radar, medical devices, or vibration sensors.

The 56800 16-bit DSP Core: **56800 16-bit DSP Core**, Program Flash, Program RAM, Data RAM, Boot Flash, Clock Generation (PLL / Oscillator), Interrupt Controller, GPIO, SCI, SPI, Quad Timer, PWM, ADC, CAN (larger devices only). Detailed specification in page 1-16 56F80x User's Manual, Table 1-2. Feature Matrix

Architecture:

Detailed information in section 1.6 figure 1-1 to 1-5 from page 1-11.

Simplified architecture:



The peripherals are different from PIC18 but the organization is almost identical.

2. Pin Diaram

Unlike PIC18, the DSP56F80x family contains several devices:

- DSP56F801
- DSP56F803
- DSP56F805
- DSP56F807

Each has different packages and pin counts. Unlike the PIC datasheet, this User's Manual does **not** present one large annotated pinout, instead it documents each signal category: Power, Ground, GPIO, ADC, PWM, SCI, SPI, CAN, Timer, Clock, Reset, JTAG.

Like PIC18, one pin often has multiple functions. Detailed functional Group information in table 2-1 page 2-3. Pin diagrams from figure 2-1 to figure 2-5.

3. Memory Organization

Memory Capacity Calculation

- **Total Words:** 8,188 words.
- **Bits per Word:** 16 bits.
- **Total Bits:** $8,188 \times 16 = 131,008$ bits.
- **Total Bytes:** $131,008 \div 8 = 16,376$ bytes

Source: Table 1-2, page 1-16:

56F801 has:

Program Flash: $8188 \times 16 \approx 16,376$ bytes

Data Flash: $2K \times 16 = 4000$ bytes

Program RAM: $1K \times 16 = 2000$ bytes

Data RAM: $1K \times 16 = 2000$ bytes

Boot Flash: $2K \times 16 = 4000$ bytes

PIC 18 memory only has: Flash, RAM, EEPROM.

DSP56F80x family has:

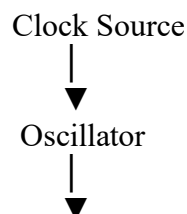
- Program Flash: Stores the main application code and instructions similar to PIC18 Flash.
- Program RAM: Holds active code that runs dynamically for faster execution. That means executable code can also exist in RAM, not only Flash like PIC18.
- Data RAM: Stores temporary variables used while the device runs similar to PIC18 Ram.
- Data Flash: Saves logs, calibration values, or user data safely, in this specific device architecture, Data Flash must replace the role of the EEPROM.
- Boot Flash: Contains low-level startup code to boot the device. Instead of immediately jumping into application like PIC18, the processor can first execute code stored inside Boot Flash.

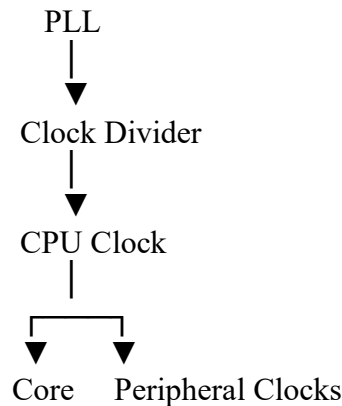
4. Oscillator / Clock

The DSP56F80x supports Internal Relaxation Oscillator or External Crystal or both depends on model. (Table 1-2, Page 1-16)

Source: Chapter 15 page 15-3 onward

The manual (Chapter 15 – On-Chip Clock Synthesis (OCCS)) describes clock generation module:





Compared to the PIC18, the DSP56F80x has a more complete and flexible Clock generation system, allowing separate control of the clock of the processor core and peripherals.

5. GPIO / I/O Ports

Source: Chapter 7 page 7-3 onward

The GPIO chapter explains

- Pin descriptions
- Register definitions
- Direction control
- Data register
- Input register

Instead of the programming model is Configure -> Write -> Read like PIC18

TRIS -> LAT -> PORT. DSP56F80x:

- Data Direction Register (DDR): Configure input/output
 - Data Register (DDR): Read or write pin state
 - Peripheral Enable Register (PER): determines the GPIO's configuration. When the PER value is one, the peripheral manages the GPIO pin. When the PER value is zero, the DDR determines the direction of data flow
- ⇒ DR serves both reading and writing unlike LAT for writing, PORT for reading.

6. Startup hardware settings

Unlike PIC18 with configuration bits, **DSP56F80x** family handle start up hardware settings in split mechanism:

PIC18 MCU Feature	PIC18 Implementation	DSP56F80x Equivalent Feature	DSP56F80x Implementation
Oscillator Selection (FOSC)	Programmed into non-volatile Configuration Word fuses.	Clock Generation (CLKGEN)	Controlled via the software-writable CLKO Select Register (CLKOSR) at runtime.

Watchdog Timer (WDTEN)	Enabled or disabled permanently via Flash config bits.	Computer Operating Properly (COP)	Configured dynamically via software using the COP Control Register (COPCTL) .
Boot Mode Selection	Controlled via configuration settings or dedicated logic.	Mode Select Pin (EXTBOOT)	Hardware logic sampling the physical EXTBOOT pin at the exact moment of a hardware Reset to latch the operating mode (like Mode 0A or 0B) into the OMR register.

7. DSP Core

Source: Figure 1-1 Page 1-11

The DSP Core is the central processing unit of the DSP56F80x family. Unlike a traditional microcontroller CPU, the DSP core contains several specialized hardware units that work together to efficiently execute digital signal processing (DSP) algorithms. By allowing multiple operations to occur in parallel, these units improve execution speed and reduce the number of instructions required for repetitive mathematical calculations.

7.1. Multiply-Accumulate (MAC) Unit

The MAC unit is dedicated hardware that performs multiplication and addition in a single operation. It is optimized for calculations commonly used in digital filters, motor control, and signal processing.

Example:

```
sum += sample * coefficient;
```

Instead of executing separate multiply and add instructions, the MAC unit combines them into one efficient operation, significantly improving performance.

7.2. 36-bit Accumulators

The DSP core contains two 36-bit accumulators (Accumulator A and Accumulator B) that store intermediate calculation results.

Although the processor is a 16-bit device, multiplication of two 16-bit values produces a much larger result. The 36-bit accumulators provide additional precision and reduce the risk of overflow during repeated mathematical operations.

7.3. Three 16-bit Input Registers

The DSP core includes three 16-bit input registers that temporarily hold operands before they are processed by the execution units.

Having multiple input registers allows the processor to access several operands simultaneously, reducing memory accesses and increasing execution efficiency during arithmetic operations.

7.4. Address Generation Unit (AGU)

The Address Generation Unit (AGU) is dedicated hardware responsible for calculating memory addresses.

Instead of using the arithmetic unit to repeatedly increment pointers or calculate array indices, the AGU automatically updates memory addresses while the execution unit performs calculations. This allows memory access and computation to occur in parallel.

7.5. Hardware Loop Unit

The Hardware Loop Unit executes repetitive loops without requiring explicit software branch instructions.

Instead of repeatedly incrementing a loop counter and testing the loop condition, the hardware automatically controls the loop execution. This reduces instruction overhead and improves performance for algorithms that repeatedly process arrays or data buffers.

Summary

Compared with a conventional microcontroller such as the PIC18F45K50, the DSP56F80x DSP core includes several dedicated hardware blocks that work simultaneously. These hardware features enable the DSP56F80x to perform mathematical computations and real-time control algorithms much more efficiently than a general-purpose microcontroller.

8. Practice

8.1. Development Environment

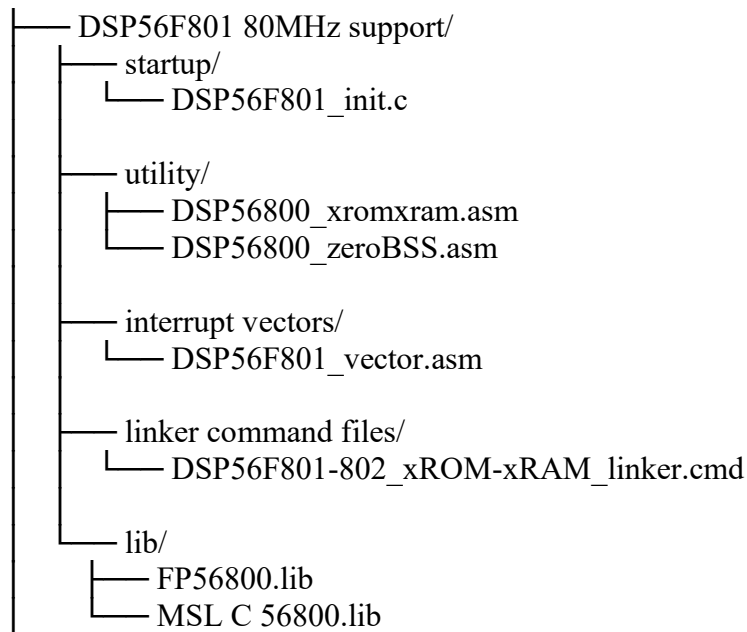
- CodeWarrior Development Studio for 56800/E Digital Signal Controllers v8.3
- CodeWarrior IDE 5.9.0 Build 4713
- **Target device:** DSP56F801
- **Stationery:** DSP56800xEEABI
- **Device configuration:** DSP56F801_80MHz
- Project type: Simple_Asm_and_C
- Alt + F7 -> Debugger -> Remote Debugging Connection: 56800 Simulator

8.2. Project structure

Create DSP56800xEEABI Stationery Project name: “DSP56F80X”

DSP56F801_80MHz is chosen for this demo project with simple_Asm_and_C option.

```
DSP56800x/
|
|— code/
|   |
|   |— main.c
|   |— MyAsm.asm
```



Folder Explanation

1. code/

code/

main.c

MyAsm.asm

This folder contains the application source code written by the developer.

main.c

This is the main program entry point.

After the processor completes its startup sequence, execution is transferred to main() where the application begins.

Typical responsibilities include:

- hardware initialization
- peripheral configuration
- application logic
- infinite main loop

Equivalent in the PIC18 project:

main.c

MyAsm.asm

This file contains user-written assembly code.

The DSP56F801 supports both C and DSP56800E assembly language.

Assembly functions can be called directly from C when higher execution speed or access to DSP-specific instructions is required.

For simple applications this file may remain empty.

2. startup/

startup/

DSP56F801_init.c

This file performs processor initialization before the application starts.

Typical initialization tasks include:

- configuring the system clock
- initializing hardware modules
- preparing memory
- enabling required peripherals

Unlike the PIC18, many device settings are configured here rather than through #pragma config.

Execution flow:

```
Power On
  ↓
Reset
  ↓
Startup Code
  ↓
DSP56F801_init.c
  ↓
main()
```

3. utility/

utility/

DSP56800_xromxram.asm

DSP56800_zeroBSS.asm

These files are automatically generated runtime support routines.

DSP56800_xromxram.asm

Responsible for copying initialized variables from Program Flash (ROM) into RAM during startup.

Example:

```
int counter = 5;
```

The initial value 5 is stored in Flash.

During startup this file copies the value into RAM so that counter contains the correct value before main() executes.

DSP56800_zeroBSS.asm

Responsible for clearing the .bss section.

The .bss section contains uninitialized global and static variables.

Example:

```
int counter;
```

```
static int value;
```

Before entering main(), this routine automatically sets these variables to zero, ensuring that the program starts from a known state.

4. interrupt vectors/

interrupt vectors/

DSP56F801_vector.asm

This file defines the interrupt vector table.

The vector table contains the addresses of interrupt service routines (ISRs).

When an interrupt occurs, the CPU automatically jumps to the corresponding vector.
Examples include:

- Reset
- Timer Interrupt
- SCI (UART)
- ADC
- PWM

This serves the same purpose as the interrupt vectors found in the PIC18 architecture.

5. linker command files/

linker command files/

DSP56F801-802_xROM-xRAM_linker.cmd

This is the linker command file.

It defines how the program is placed into memory.

Typical information includes:

- Program Flash location
- RAM location
- Stack size
- Heap size
- Interrupt vector addresses
- Memory sections

Conceptually similar to the PIC18 memory map, but much more flexible.

The linker uses this file to determine where every function and variable should reside in memory.

6. lib/

lib/

FP56800.lib

MSL C 56800.lib

These are precompiled runtime libraries provided by CodeWarrior.

FP56800.lib

Provides floating-point mathematical functions.

Examples include:

- multiplication
- division
- trigonometric functions
- square root

These functions are linked automatically when required by the application.

MSL C 56800.lib

Motorola Standard Library (MSL).

Equivalent to the standard C runtime library.

Provides common functions such as:

- memory operations
- string manipulation
- runtime initialization

- standard C support

Complete Startup Sequence

The complete execution sequence of a DSP56F801 application is shown below.

Power Applied



Processor Reset



Interrupt Vector Table
(DSP56F801_vector.asm)



Runtime Startup



Initialize Hardware
(DSP56F801_init.c)



Copy Initialized Data
(DSP56800_xromxram.asm)



Clear .bss Variables
(DSP56800_zeroBSS.asm)



main()



Application Loop

8.3. Write the program

Project: DSP56F801 DSP + GPIO Demonstration

The purpose of this project is to demonstrate both major aspects of the DSP56F801:

1. **Microcontroller-style peripheral control**
 - Configure a GPIO port.
 - Configure a GPIO pin as an output.
 - Write a digital output value to the GPIO register.
2. **DSP-style numerical computation**
 - Pass numerical operands from C to an assembly routine.

- Use the DSP56800 Data ALU and its hardware Multiply-Accumulate (MAC) unit.
 - Return the calculated result to C.
3. **Integration of both capabilities**
- Perform a DSP calculation.
 - Use the calculation result to control a GPIO output.

The application therefore follows this sequence:

Initialize GPIO



Prepare DSP operands



Call `dsp_mac()`



Execute hardware MAC



Return result to C



Check result



Set GPIO HIGH or LOW



Wait in infinite loop

8.3.1. GPIO Register Access

Unlike the PIC18F45K50 project, this project does not use XC8 register definitions such as:
`TRISBbits.TRISB0`
`LATBbits.LATB0`

The DSP56F801 peripheral registers are memory-mapped. The GPIOA register block is located at address `$0FB0`. Source:

The program therefore defines the GPIO registers explicitly using volatile pointers:
`typedef unsigned short UWord16;`

```
#define GPIOA_BASE 0x0FB0
```

```
#define GPIO_A_DR (*(volatile UWord16 *)(GPIOA_BASE + 1))
#define GPIO_A_DDR (*(volatile UWord16 *)(GPIOA_BASE + 2))
#define GPIO_A_PER (*(volatile UWord16 *)(GPIOA_BASE + 3))
```

Source: *Table 3-34. GPIO Port A Registers Address Map; Table 7-7. GPIO Memory Map*

The volatile qualifier is important because these memory locations represent hardware registers rather than ordinary program variables. A read or write to the address must correspond to an actual hardware access.

GPIO Initialization

The application uses PA0 as a digital output.

```
static void gpio_init(void)
{
    GPIO_A_PER = 0x0000;
    GPIO_A_DDR = 0x0001;
    GPIO_A_DR = 0x0000;
}
```

The initialization consists of three steps.

Select GPIO functionality

```
GPIO_A_PER = 0x0000;
```

This selects GPIO functionality for the corresponding pins instead of assigning the pins to their alternative peripheral functions.

Configure PA0 as an output

```
GPIO_A_DDR = 0x0001;
```

Bit 0 is set, therefore PA0 is configured as an output.

GPIO Output Functions

Two small functions are used to make the peripheral operation easier to understand:

```
static void gpio_set_high(void)
{
    GPIO_A_DR = 0x0001;
}
```

```
static void gpio_set_low(void)
{
    GPIO_A_DR = 0x0000;
}
```

Therefore:

```
gpio_set_high()
```

↓

GPIO_A_DR bit 0 = 1

↓

PA0 (Port A) HIGH

and:

```
gpio_set_low()
```

↓

GPIO_A_DR bit 0 = 0

↓

PA0 (Port A) LOW

This is the microcontroller-style portion of the application.

8.3.2. DSP Core Demonstration

The DSP portion uses a separate assembly function:

```
int dsp_mac(int sample, int coefficient);
```

The C code calls this function:

```
result = dsp_mac(sample, coefficient);
```

The CodeWarrior DSP56800 calling convention specifies that the first two 16-bit values passed to a function are placed in:

First 16-bit argument Y0

Second 16-bit argument Y1

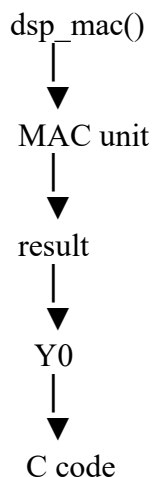
A 16-bit non-address return value is returned in:

Y0

Therefore, this function is mapped as:

C		DSP register
---	--	--------------

sample	—————▶	Y0
coefficient	—————▶	Y1



This calling convention is documented in the CodeWarrior **56800/E Digital Signal Controllers: DSP56F80x/DSP56F82x Targeting Manual**, in the section covering C for DSP56800 and calling conventions. Link: https://www.nxp.com/docs/en/user-guide/Targeting_56800.pdf

Q15 Input Values

The example uses fixed-point fractional values:

```
int sample = 0x4000;
```

```
int coefficient = 0x2000;
```

These represent approximately:

0x4000 → 0.5

0x2000 → 0.25

Therefore the intended mathematical operation is:

$0.5 \times 0.25 = 0.125$

This representation is appropriate for demonstrating the DSP56800 because the architecture provides dedicated fractional arithmetic in its Data ALU.

The DSP56800 Family Manual explains that the Data ALU supports both fractional and integer arithmetic. It also describes the location of the binary point for fractional operands and the 36-bit accumulator representation.

DSP56800 Data ALU

The DSP56800 core contains a dedicated Data Arithmetic Logic Unit (Data ALU).

Important components include:

Three 16-bit input registers

X0

Y0

Y1

Two 36-bit accumulators

A

B

Multiply-Accumulate unit

MAC

The DSP56800 Family Manual describes the Data ALU as containing three 16-bit input registers, two 36-bit accumulators, and a parallel single-cycle MAC unit.

The accumulators are larger than the 16-bit input operands:

$A = A_2 : A_1 : A_0$

4 16 16 bits

giving a total of:

$4 + 16 + 16 = 36$ bits

This larger accumulator allows intermediate DSP calculations to retain more precision.

Assembly Implementation of dsp_mac()

The assembly function is:

```
section    rtlib
```

```
org       p:
```

```
global    Fdsp_mac
```

Fdsp_mac:

```
move      Y0,X0
```

```
clr       A
```

```
MAC       Y1,X0,A
```

```
move      A1,Y0
```

```
rts
```

endsec

The function performs the following operations.

Move the first operand to X0

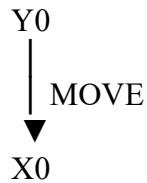
move Y0,X0

The C calling convention provides:

Y0 = sample

Y1 = coefficient

The MAC instruction can use X0 and Y1 as its two source operands, so the first operand is moved:



Clear the Accumulator

clr A

This initializes the 36-bit accumulator:

$A = 0$

The purpose is to implement:

$A = 0 + \text{sample} \times \text{coefficient}$

The CLR instruction operates on the complete 36-bit accumulator. The DSP56800 Family Manual describes CLR F as clearing a 36-bit accumulator.

Execute the Hardware MAC

The key instruction is:

MAC Y1,X0,A

Conceptually, this performs:

$A = A + Y1 \times X0$

Because the accumulator was previously cleared:

$A = 0 + \text{coefficient} \times \text{sample}$

Therefore:

$A = \text{coefficient} \times \text{sample}$

This is not a software implementation using separate multiplication and addition instructions.

The DSP56800 has dedicated MAC hardware.

The DSP56800 Family Manual explicitly lists:

MAC Y1,X0,A

as a valid MAC instruction form and describes the instruction as a fractional multiply-accumulate operation.

The manual also states that the multiply-accumulate operation is a **single-cycle operation** and that the multiplier supports 16-bit $\times$ 16-bit multiplication.

Retrieve the Result

The accumulator is 36 bits:

$A = A2 : A1 : A0$

where:

A2 = extension

A1 = most significant portion (MSP)

A0 = least significant portion (LSP)

For this simple 16-bit Q15 demonstration, the 16-bit result is taken from the appropriate accumulator result portion:

move A1,Y0

and then:

Y0 → C result

The DSP56800 Family Manual explains that A1 is the 16-bit MSP of accumulator A and that individual accumulator portions can be accessed separately. It also distinguishes this from reading the entire accumulator, which enables data limiting/saturation.

For production DSP code where the result must be safely converted to a 16-bit signal, the complete accumulator A can be used as the source of a MOVE so that the DSP's data limiter can perform saturation when required.

Return to C

The final assembly instruction before returning is:

move A1,Y0

The result is therefore placed in:

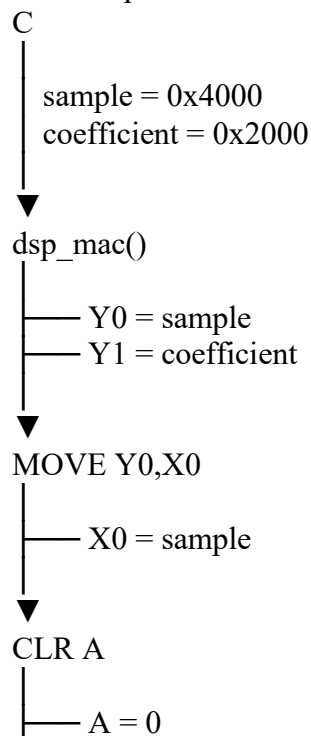
Y0

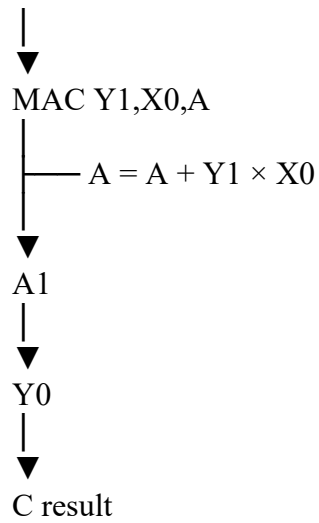
The RTS instruction then returns to the C caller:

rts

The CodeWarrior calling convention specifies that a 16-bit non-address function result is returned in Y0.

The complete data flow is therefore:





8.3.3. Integration With GPIO

After the DSP calculation:

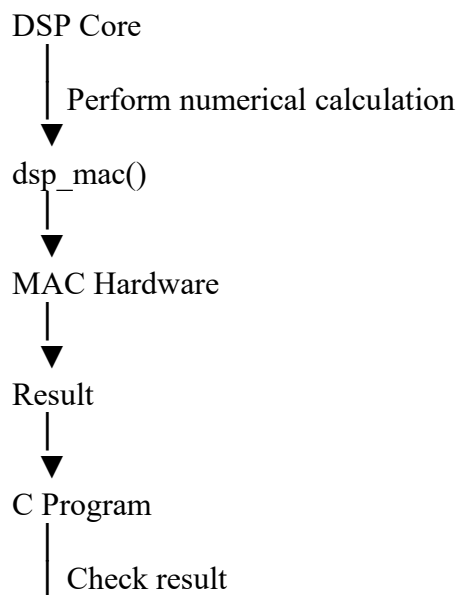
`result = dsp_mac(sample, coefficient);`

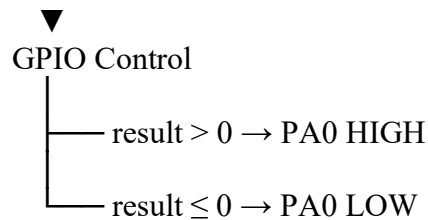
the C program uses the result:

```

if (result > 0)
{
    gpio_set_high();
}
else
{
    gpio_set_low();
}
  
```

Overall flow:





This demonstrates the interaction between the DSP core and conventional MCU-style peripherals.

8.4 Build, Debugging, and Memory Verification

8.4.1. Build Output

The DSP56F801 project was built using CodeWarrior Development Studio for 56800/E.

The build generated the following main output files:

internal_xROM-xRAM.elf

internal_xROM-xRAM.elf.xMAP

The generated ELF file contains the executable image and debugging information required by the CodeWarrior debugger.

The .xMAP file provides information about how the linker placed program code, data, variables, stack, heap, and other sections into the DSP56F801 memory spaces.

8.4.2. Debugging Procedure

The debugging procedure was designed to verify the same concepts previously examined on the PIC18F45K50.

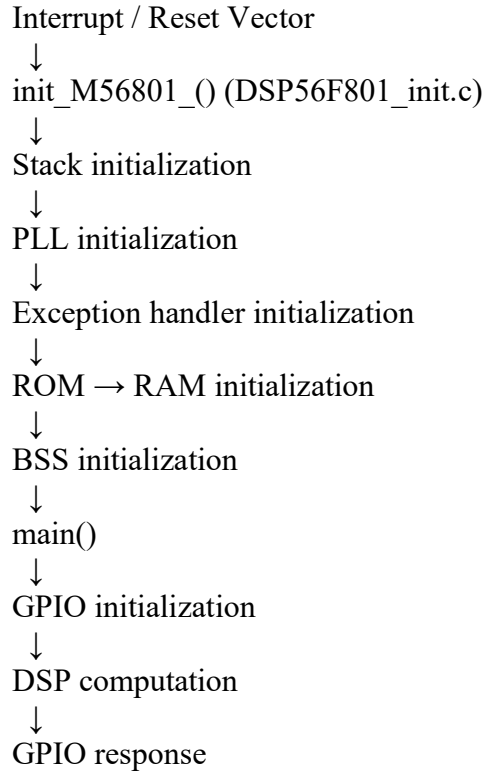
The following steps were performed:

1. Build the project successfully.
2. Start the CodeWarrior debugger.
3. Load the generated:
internal_xROM-xRAM.elf
4. Set a breakpoint at the beginning of main().
5. Start program execution.
6. Verify that execution stops at the breakpoint.
7. Use **Step Into** / **Step Over** to execute individual instructions.
8. Observe the program counter and register values.
9. Inspect the call stack when entering functions such as:
 - gpio_init()
 - gpio_set_high()
 - gpio_set_low()
 - dsp_mac()
10. Open the disassembly window to compare C code with the generated DSP56800 assembly instructions.

The debugging flow can be summarized as:

Reset





8.4.3. Verifying the DSP Core Through Debugging

The demonstration program contains both MCU-style peripheral control and DSP-style computation.

The MCU-style portion performs GPIO configuration:

```
gpio_init();
```

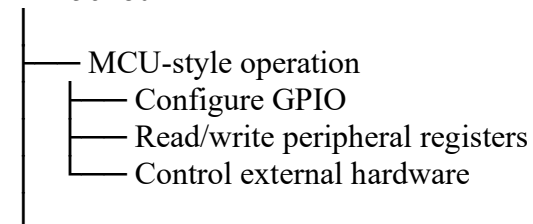
The DSP-style portion performs the multiply-accumulate operation:

```
result = dsp_mac(sample, coefficient);
```

The result is then used by the MCU-style code:

```
if (result > 0)
{
    gpio_set_high();
}
else
{
    gpio_set_low();
}
```

This demonstrates the two major roles of the DSP56F801:
DSP56F801



- └─ DSP-style operation
 - └─ Multiply
 - └─ Accumulate
 - └─ Use DSP registers
 - └─ Execute DSP-oriented arithmetic

8.4.4. Memory Organization

Unlike a conventional single-address-space microcontroller, the DSP56F801 uses separate program and data memory spaces.

The .xMAP file identifies these spaces separately.

The relevant memory regions in this project are:

Program memory (P memory)

- └─ Interrupt vectors
- └─ Program Flash
- └─ Program RAM
- └─ Boot Flash

Data memory (X memory)

- └─ Compiler registers
- └─ Internal RAM
- └─ Peripheral registers
- └─ Data Flash

The linker configuration defines:

Program Flash:

ORIGIN = 0x0080
LENGTH = 0x1F80

Internal Program RAM:

ORIGIN = 0x7C00
LENGTH = 0x0400

Internal X RAM:

ORIGIN = 0x0040
LENGTH = 0x03C0

Peripheral space:

ORIGIN = 0x0C00
LENGTH = 0x0400

X Flash:

ORIGIN = 0x1000
LENGTH = 0x0800

8.4.5. Important .xMAP Information

The .xMAP file for the final project contains the following program layout:

.executing_code

```

00000080 0000000A Fgpio_init
0000008A 00000004 Fgpio_set_high
0000008E 00000004 Fgpio_set_low
00000092 0000001C Fmain
000000AE 0000003D Finit_M56801_
000000EB 0000000A F__console_write
000000F5 0000000A Ffflush
000000FF 00000006 Fflush_buf
00000105 00000009 rtlb.text
00000109 00000000 Fdsp_mac
00000116 00000012 F__romCopy
00000128 0000000C F__bssClear

```

The most important entries are:

```

P:$0080  gpio_init()
P:$008A  gpio_set_high()
P:$008E  gpio_set_low()
P:$0092  main()
P:$00AE  init_M56801_()
P:$0109  dsp_mac()

```

This allows the developer to correlate functions in the C/assembly source with their actual locations in program memory.

For example:

```

main.c
  main()
    ↓
P:$0092

```

```

MyAsm.asm
  dsp_mac()
    ↓
P:$0109

```

This is useful when debugging at the assembly level.

8.4.6 Interrupt Vector Memory

The .xMAP file shows:

```
.interrupt_vectors
```

```

P:$0004
size = $007C
and:
.interrupt_vectors_mirror

```

```

P:$8000
size = $0004

```

The linker therefore places the interrupt-vector section at the beginning of program memory, while also providing the corresponding mirror section in Boot Flash. The map confirms that the vector section is included in the final executable.

8.4.7 Program Code Memory Usage

The complete executable code occupies:

P:\$0080 → P:\$0134

with:

size = \$00B4

Therefore, the application and startup code occupy only a small portion of the available program Flash.

The important observation is not only the amount of memory used, but **where each section is located**.

8.4.8 Data Memory and ROM-to-RAM Initialization

The project uses an **xROM-to-xRAM** memory model.

The linker places initialized data in Flash as its resident/load location and specifies an X-RAM address as its runtime location.

The .xMAP file reports:

F_data_ROM_addr = \$1000

F_data_RAM_addr = \$0040

F_data_size = \$0002

Therefore:

Flash/X-ROM

X:\$1000

↓ ROM → RAM copy during startup

X:\$0040

Internal X-RAM

The startup code contains:

jsr __romCopy

jsr __bssClear

Therefore, before main() executes, the runtime environment initializes RAM data and clears the BSS section.

This demonstrates an important embedded-system concept:

Data initialized in the program image does not necessarily execute directly from its final RAM location. The startup code can copy the initial values from non-volatile memory into RAM before main() starts.

8.4.9. Heap and Stack

The .xMAP file also provides the memory allocated for dynamic memory and the stack.

Heap:

start = \$0094

size = \$0100

end = \$0194

Stack:

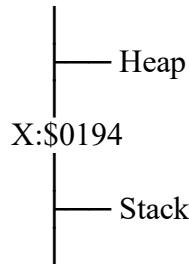
start = \$0194

size = \$0200

end = \$0394

Therefore:

X:\$0094



X:\$0394

The startup code uses the linker-generated stack address:

```
move #_stack_addr,r0
```

```
move r0,sp
```

This demonstrates how the linker, startup assembly, and C runtime cooperate to establish the execution environment before main().

8.4.10 Peripheral Memory

The linker reserves the peripheral address range:

.x_peripherals

X:\$0C00

size = \$0400

Therefore:

X:\$0C00 – X:\$0FFF

is reserved for peripheral registers.

The GPIOA registers used in the project are located in the memory-mapped peripheral region.

For GPIOA:

X:\$0FB0 PUR

X:\$0FB1 DR

X:\$0FB2 DDR

X:\$0FB3 PER

The C program accesses these registers through volatile memory-mapped pointers.

This demonstrates how software can control MCU-style peripherals using the DSP56F801's X data memory space.

For this project, the most useful entries to remember are:

P:\$0092 main()

P:\$0109 dsp_mac()

X:\$0040 runtime initialized data

X:\$0094 heap
X:\$0194 stack

X:\$0C00 peripheral region
X:\$1000 X-ROM data

8.4.11. Conclusion

The program is ready to be flashed.

If Hardware Were Available

The procedure would be:

1. Connect DSP56F801 development board
↓
2. Connect compatible CodeWarrior debug/programming interface
↓
3. Power the target
↓
4. Select the correct DSP56F801 target
↓
5. Build the project
↓
6. Start Debug/Program
↓
7. CodeWarrior loads the executable
↓
8. Program Flash is programmed
↓
9. Reset/run the DSP
↓
10. Verify program operation